CSS

Page Layout

practical use of CSS

Spolufinancováno Evropskou unií

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

doc. Ing. Radek Burget, Ph.D.

burgetr@vut.cz

Ing. Jiří Hynek, Ph.D.

hynek@vut.cz

Layout

- **Layout** = visual and logical organization of a page
- Division of the page to logical units and their positioning
- Goals
  - Clarity of the content
  - Ease of use
  - Visual appeal
- Means
  - HTML Frames (obsolete, many disadvantages)
  - HTML Tables (obsolete)
  - CSS

Table layout

- Misusing tables for defining a layout
- Historically the only solution, now not used anymore
- Advantages
  - Simple design
  - Compatibility
  - Many options in content manipulation, alignment, …
- Disadvantages
  - Page layout is defined directly in HTML
  - Long and obscure code (`<tr><td><tr><td>`)
  - The table is displayed after being read completely (slow)

Table Layout – Example 1

Table Layout – Example 2

CSS Layout

- Visual blocks are created mostly using the sectioning HTML elements
  - Semantic elements `<header>`, `<footer>`, `<article>`, …
  - The `<div>` element for remaining parts
  - The `id` and `class` attributes
- The positions of the elements are defined by CSS
  - Width and height
  - Margins
  - Floating blocks
  - Positioning (absolute or relative)
  - Flex and grid layout

Page width

- Fixed width × flexible
- Design aspects
  - Wide range of browsing devices
    - Desktop even 2560px, mobile phone 320px
  - It's inconvenient to read long text lines
    - Use narrow lines
    - Greater font
    - More columns
- Currently, we prefer flexible width

Page setup

- HTML `<div>` container

```html
<body>
    <div id="page">
        … page content ….
    </div>
</body>
```

- Flexible width

```css
body {
    margin: 0; padding: 1em;
}
#page {
    width: 80%;
    margin: auto;
    padding: 1em;
}
```

- Fixed width

```css
body {
    margin: 0; padding: 1em;
}
#page {
    width: 1000px;
    margin: auto;
    padding: 1em;
}
```

Page Width – Example

Positioning the elements side-by-side

- With tables: for small local areas
  - Fixed small parts of the page, form fields, …
  - Easy to set up
- Absolute positioning
- Floating objects
- Special `display:` values (e.g. inline-block)
- Flexbox
- Grid layout

Absolute positioning

- Left element has absolute position
- Right element has a right margin

```html
<div id="left">Left</div>
<div id="right">Right</div>
```

```css
#left {
    width: 5em;
    position: absolute;
    left: 0; top: 0;
}
#right {
    margin-left: 5em;
}
```

LeftRight

Absolute positioning – Disadvantages

- How to determine the vertical position?
- Absolutely positioned element may overlap with other content

```html
<p>Preceding paragraph</p>
<div id="left">Left</div>
<div id="right">Right</div>
<p>Following paragraph</p>
```

```css
#left {
    width: 5em;
    position: absolute;
    left: 0; top: 0.5em;
}
#right {
    margin-left: 5em;
}
```

Preceding paragraph LeftRight Following par

Absolute positioning – Container

- Absolute positioning within a parent element (container) with position: relative

```html
<p>Preceding paragraph</p>
<div class="columns">
  <div id="left">Left</div>
  <div id="right">Right</div>
</div>
<p>Following paragraph</p>
```

```css
.columns {
    position: relative;
}
#left {
    width: 5em;
    position: absolute;
    left: 0; top: 0em;
}
#right {
    margin-left: 5em;
}
```

Preceding paragraph Following paragra

Left

Right

What if we need more than two columns?

Floating Blocks

- All columns have `float: left`

```html
<p>Preceding paragraph</p>
<div id="left" class="col">Left</div>
<div id="right" class="col">Right</div>
<p>Following paragraph</p>
```

```css
.col {
    width: 5em;
    float: left;
}
```

Preceding paragraph Left Right Following

Floating Blocks with `clear`

- Use `clear: left` or `both` for the following paragraph

```
<p>Preceding paragraph</p>
<div id="left" class="col">Left</div>
<div id="right" class="col">Right</div>
<p class="clear">Following paragraph</p
```

```
.col {
    width: 5em;
    float: left;
}
.clear {
    clear: both;
}
```

Preceding paragraph Left Right Following

Floating Blocks – Container

- The enclosing block `.columns` remains empty

```
<p>Preceding paragraph</p>
<div class="columns">
  <div id="left" class="col">Left</div>
  <div id="right" class="col">Right</di
</div>
<p>Following paragraph</p>
```

```
.col {
    width: 5em;
    float: left;
}
.columns {
    clear: both;
    border: 5px solid black;
}
```

Preceding paragraph Following paragra

Left

Right

Floating Blocks – Container with overflow

- With `overflow: hidden`, the block `.columns` encloses all floating blocks

```
<p>Preceding paragraph</p>
<div class="columns">
  <div id="left" class="col">Left</div>
  <div id="right" class="col">Right</di
</div>
<p>Following paragraph</p>
```

```
.col {
    width: 5em;
    float: left;
}
.columns {
    clear: both;
    border: 5px solid black;
    overflow: hidden;
}
```

```
Preceding paragraph      Following paragra
                   Left
                   Right
```

Multiple columns

Special `display` Property Values

- Inline block `display: inline-block`
  - Blocks positioned as inline elements
- Table cells `display: table-cell`
  - They create an anonymous table
  - Similarly `table-row`, `table`, `inline-table`

Special `display` Property Values – Examples

Flexbox

- Mostly for local layouts
  - Forms, list items, …
- Parent element is used as a **flex container**
  - By setting `display: flex`
  - We define the `main axis` and the `cross axis`
- The child elements become the **flex items**

Flex Container Properties

- flex-direction – main axis direction
- flex-wrap – content wrapping to more rows/columns
- justify-content – content justification in the row/column
- align-items – item alignment

Item Properties

- flex: `<flex-grow> <flex-shrink> <flex-basis>`
- `flex-grow` – how the item grows when there is more space available
  - `flex-grow: 0` – does not grow at all
  - `flex-grow: 2` – weight 2, relative to the remaining weights
- `flex-shrink` – how the item shrinks when there is not enough space
  - `flex-shrink: 1`
- `flex-basis` – initial main size
  - `flex-basis: auto`
  - `flex-basis: content`
  - `flex-basis: 10em`
  - `flex-basis: 40%`

Grid layout

- Parent element is the **grid container**
  - By setting `display: grid`
  - We set the number of grid rows and columns
    - Explicit (template) nebo implicit (auto)
    - The `fr` (fraction) length unit
  - Named areas may be created (multiple grid cells)
- Grid position is set for child elements
  - `grid-row-start`, `grid-row-end` (`grid-row`)
  - `grid-column-start`, `grid-column-end` (`grid-column`)
  - or `grid-area` (demo)
- Description and demo on MDN
- Detailed description

Selected CSS Solutions

Using Icons

```html
<a href="#">Click me</a>
```

```css
a {
    background: #eeeeff url('newwin32.png') top right no-repeat;
    padding-right: 35px;
    padding-top: 35px;
}
```

Click me

Horizontal list

```html
<ul>
  <li><a href="#">Item 1</a></li>
  <li><a href="#">Item 2</a></li>
  <li><a href="#">Item 3</a></li>
</ul>
```

- Item 1
- Item 2
- Item 3

Horizontal list

```
ul.vertlist1 li { display: inline; }
```

```
<ul class="vertlist">
  <li><a href="#">Item 1</a></li>
  <li><a href="#">Item 2</a></li>
  <li><a href="#">Item 3</a></li>
</ul>
```

- Item 1
- Item 2
- Item 3

Horizontal list

```
ul.vertlist1 li { display: inline-block; }
```

```
<ul class="vertlist">
  <li><a href="#">Item 1</a></li>
  <li><a href="#">Item 2</a></li>
  <li><a href="#">Item 3</a></li>
</ul>
```

- Item 1
- Item 2
- Item 3

Image sprites

- More images in one larger image
- The individual images are extracted using the background positioning
- More efficient and smooth user experience
- Image sprite 1
- Image sprite 2

Image sprites (II)

- Example 1

```
a.smile { background: url("imageSprite.png") -315px -28px; }
a.smile:hover { background: url("imageSprite.png") -341px -28px; }

a.del { background: url("imageSprite.png") -269px -52px; }
a.del:hover { background: url("imageSprite.png") -269px -68px; }
```

- Example 2

CSS Usage Tips

External Style Definitions

- Use external style definitions
  - Shared style sheets for a set of documents
- Multiple style sheets may be used

```
<link ="" rel="stylesheet" src="common.css" type="text/css"/>
<link ="" rel="stylesheet" src="section.css" type="text/css"/>
```

Distinguish between `id` and `class`

- `id` attribute
  - For unique elements
  - Easier to use (more specific selector)
  - Clearer style sheet
- `class` attribute
  - For repeating elements

Class and id names

- Use descriptive class and id names
  - Should describe the **meaning** , not the appearance
  - Bad: `rightcolumn`, `redtext`
  - Correct: `menu`, `important`
- Use a small set of class names
  - Many names are hard to remember
  - Use context selectors

```
p span.first {…} /* first letter of the paragraph */
p.first {…} /* first paragraph in the article */
li.first {…} /* first list item */
```

Context Selectors

- Don't create useless classes, use context selectors
- Wrong:

```html
<h2 class="articleheading">Heading</h2>
<p class="articlepar">…</p>
 …
<p class="articlepar">…</p>
```

```css
.articleheading { … }
.articlepar { … }
```

Context Selectors

- Right:

```html
<div class="article">
   <h2>Title</h2>
   <p>…</p>
   …
   <p>…</p>
</div>
```

```css
.article h2 { … }
.article p { … }
```

Context Selectors

- Wrong:

```
<ul>
<li class="menu">…</li>
<li class="menu">…</li>
 …
<li class="menu">…</li>
</ul>
```

```
.menu { … }
```

Context Selectors

- Right:

```
<ul class="menu">
<li>…</li>
<li>…</li>
 …
<li>…</li>
</ul>
```

```
ul.menu li { … }
```

Combine Classes

- Use more classes for one element
  - `class="importantmessage"` vs. `class="message important"`
- Cascade the rules

```
h1, h2 {
   font-family: "Arial CE", Arial, sans-serif;
   font-weight: bold;
}
h1 {
   font-size: 200%
}
h2 {
   font-size: 160%
}
```

Writing the Style Sheet

- Write with a clear style
- Choose some selector order
  - Elements, id, classes, …
  - Or according to the page organization
- Choose some property specification order
  - E.g. sizes, margins, position, colors and borders
- Write rules to separate lines
  - Easy to copy
- Use comments `/* comment */`

To be continued…

CSS – Creating a complex layout